

CGV SPRINT 1 // PROJECT GUIDE

FRACTURE RUN

A three-level Three.js browser game inspired by Smash Hit and Temple Run

BREAK THE PATH. REACH THE LIFT.

Working document - concept, levels, cameras, environments, technical plan, rubric alignment, and Sprint 1 backlog

Working title

Fracture Run

Alternative titles: Glass Ascent, Shattershift, Resonance Tower.

1. Project summary

Fracture Run is a three-level 3D browser game built with Three.js. The player travels automatically through a surreal, shifting tower and fires a limited supply of energy spheres to break glass targets, disable hazards, and open a path forward.

The core interaction is inspired by Smash Hit: forward movement, rhythmic projectile throwing, destructible glass, limited ammunition, and a minimalist audiovisual world. Temple Run contributes the third-person chase camera, lane decisions, turns, and obstacle anticipation. Smash Hit remains the primary gameplay and environmental reference; Temple Run contributes selected camera and movement ideas without changing the game's central identity.

Reference games

- Smash Hit - primary reference: <https://www.youtube.com/watch?v=XL5as-w7Q1g>
- Temple Run 2 - secondary reference: <https://www.youtube.com/watch?v=fuQf-iGCmKA>

Each level occupies a different layer of the same tower. A glass elevator connects the levels and acts as a playable transition rather than a loading screen. The tower is failing, and the player must reach the control core at the top before its three sectors collapse.

2. Design goals

- Create a complete, polished game that can be played from start to finish in about 6-10 minutes.
- Make throwing and shattering objects feel satisfying through physics, particles, lighting, sound, and camera feedback.
- Use several camera modes for gameplay purposes, not only to satisfy the rubric.
- Give every level a distinct mechanic, visual identity, and kind of challenge.
- Keep the scope realistic for a short Three.js project and modest lab hardware.
- Make the team's own technical work visible through custom shaders, procedural layouts, original mechanics, and clear scene hierarchy.

3. Player experience

Player fantasy

The player is a runner trapped inside an unstable glass tower. They survive by breaking structures at precisely the right moment and restoring three tower systems on the way to the control core.

Core loop

- 1 Move automatically through the current sector.
- 2 Aim with the mouse and throw an energy sphere.
- 3 Break targets to gain spheres, score, or open the route.
- 4 Dodge or destroy approaching hazards using keyboard movement.
- 5 Reach the sector exit with enough integrity to continue.
- 6 Enter the elevator, review the score, and survive a short transition event.

Controls

- **Mouse movement:** aim the reticle.
- **Left mouse button:** throw an energy sphere.
- **A / D** or left / right arrows: change lane or strafe.
- **W / S** or up / down arrows: level-specific movement, such as jump/slide or vertical movement.
- **C:** cycle allowed camera modes.
- **Esc:** pause and open options.
- **R:** restart after a win or loss without refreshing the page.

The exact movement mapping should be tested early. The final controls must use both keyboard and mouse, feel responsive, and remain simple enough to learn without a tutorial wall.

4. Game rules and systems

Resources

- Spheres: each throw consumes one sphere. Breaking crystal nodes grants additional spheres.
- Integrity: collisions reduce integrity. Reaching zero ends the run.
- Score: earned through targets, near misses, accuracy, combos, and remaining spheres.
- Combo multiplier: increases when the player breaks targets without missing and decays after a short delay.

Win and loss states

- Win a level by reaching and activating its exit gate.
- Win the game by stabilising the control core at the end of Level 3.
- Lose by reaching zero integrity, falling from the route, or failing a clearly signalled timed sequence.
- The game-over screen offers restart, level retry where appropriate, settings, and credits.

Difficulty curve

Difficulty comes from combining already-learned mechanics, not simply increasing speed. Early obstacles introduce one decision at a time; later sections combine aiming, movement, timing, and camera awareness.

5. Three distinct levels

Level 1 - The Glass Causeway

Identity: a bright, abstract corridor of glass bridges, rotating panels, reflective walls, and open sky.

Camera: primarily first-person to focus attention on aiming and impact.

New mechanic: projectile throwing and destructible routes.

Challenge type: accuracy and resource management.

The player learns to aim, throw, change lanes, break blocking panes, and hit crystal nodes. Some panes are optional score targets; others must be destroyed to avoid damage. A short final sequence asks the player to break three locks in order to open the elevator.

Rubric opportunities: reflections, refraction, shadows, transparent materials, projectile physics, particles, multiple lights, and an animated environment.

Elevator 1 - Calibration Lift

The camera pulls out from first-person to a third-person orbit, briefly revealing the player's avatar and the tower below. The player can look around while the lift moves. Broken shards and score particles spiral upward around the elevator. This creates a natural camera change, masks level loading, and gives the player a short rest.

Level 2 - The Shifting Foundry

Identity: a darker mechanical sector with moving walls, pistons, rails, heat vents, and glowing industrial glass.

Camera: third-person chase camera inspired by Temple Run, with cinematic side and corner cameras at selected set pieces.

New mechanic: lane changes, jumping/sliding, and 90-degree route turns while still aiming at breakable switches.

Challenge type: movement timing and obstacle anticipation.

The player dodges solid hazards that cannot all be destroyed. Glass switches alter the track: a successful shot may retract a wall, rotate a bridge, or disable a piston. At marked junctions, the route turns and the camera eases around the player rather than snapping. The level ends with a timed escape from closing walls.

Rubric opportunities: hierarchical moving machinery, animated avatar, smooth chase camera, multiple camera angles, dynamic lighting, normal/bump maps, and meaningful 3D movement.

Elevator 2 - Gravity Fault

The second lift is damaged. Gravity weakens and spheres, shards, and the avatar float inside the cabin. The player shoots three stabilisers while the camera transitions between an interior first-person view, an exterior tower view, and a top-down orthographic diagnostic view. This previews Level 3's gravity mechanic.

Level 3 - The Inverted Core

Identity: the tower's fractured control core, suspended in a storm above the world. Platforms, glass rings, and debris rotate around a central energy reactor.

Camera: a controlled orbit/chase camera with short top-down and picture-in-picture diagnostic views.

New mechanic: gravity orientation changes. Shooting anchor nodes rotates the route or shifts gravity to a wall or ceiling.

Challenge type: spatial reasoning and a multi-stage boss/set-piece encounter.

The player moves around the central core instead of along a straight corridor. They must identify and break shield nodes from different orientations while avoiding rotating debris. The final encounter has three readable phases: expose the core, redirect its energy using anchor nodes, then shatter the unstable outer shell. Mechanics from Levels 1 and 2 return in combined form, but gravity switching and the arena structure make the experience distinct.

Rubric opportunities: full 3D navigation, custom energy shader, dynamic skybox, force-field effects, picture-in-picture, spatial audio, and a memorable final set piece.

One-sentence distinction test

- Level 1 is about aiming and deciding what to break with limited ammunition.
- Level 2 is about dodging and changing lanes while shooting switches that reshape the route.
- Level 3 is about changing gravity and viewing the arena from different orientations to solve a boss encounter.

6. Camera plan

The game should use one camera controller that blends between defined rigs to avoid abrupt cuts and duplicated logic.

Camera	Purpose	Main use
First-person	Precision aiming and immersion	Level 1
Third-person chase	Shows the avatar, lanes, and upcoming hazards	Level 2
Orbit camera	Spatial awareness around the core	Level 3
Cinematic track camera	Highlights turns, elevator travel, and set pieces	Transitions
Orthographic camera	Diagnostic/minimap view and rubric coverage	Elevator 2 / Level 3

Camera	Purpose	Main use
Picture-in-picture	Shows a switch's effect or a dangerous area	Selected Level 2/3 moments

Camera changes should be triggered by gameplay zones and blended over time. The player may cycle only between modes that are suitable for the current section, preventing a camera choice from making obstacles unfair.

7. Visual and audio direction

Visual language

- A shared tower architecture makes the game coherent.
- Level 1 uses clear glass, white structures, cyan light, and open space.
- Level 2 uses metal, warning colours, smoke/heat, and mechanical motion.
- Level 3 uses black glass, electric colour accents, a storm skybox, and unstable energy.
- Breakable objects share a recognisable material and highlight response.
- Dangerous solid objects use a different silhouette and material so players never have to guess what can be broken.

Environment progression

The surroundings change as the player climbs, while a few repeated tower motifs - octagonal frames, energy conduits, glass seams, and elevator architecture - show that all three levels belong to the same world.

Environment layer	Level 1: Glass Causeway	Level 2: Shifting Foundry	Level 3: Inverted Core
Architecture	Wide glass bridges, clean arches, floating geometric rooms	Narrow machinery bays, rail junctions, pistons, vents, moving walls	Broken rings, suspended platforms, rotating frames, exposed reactor
Surrounding world	Bright cloudscape and distant tower sections	Enclosed industrial shaft with glimpses of the city below	Open storm sky, floating debris, lightning, and the fractured tower exterior
Materials	Clear/frosted glass, white ceramic, brushed silver	Dark metal, reinforced glass, hazard paint, glowing heat panels	Black glass, iridescent metal, energy membranes, cracked stone/ceramic
Lighting	Cool daylight with soft cyan reflections	Directional work lights, red warnings, orange furnace glow	High-contrast lightning, pulsing reactor light, animated energy bands
Motion	Slowly rotating sculptures and drifting fragments	Belts, fans, pistons, shutters, steam and moving walls	Orbiting debris, rotating gravity rings, unstable platforms and force fields
Atmosphere	Calm, spacious and precise	Pressurised, loud and urgent	Unstable, exposed and climactic
Gameplay readability	Glass objects stand out against the bright sky	Breakable switches glow cyan amid solid red/metal hazards	Anchor nodes and safe surfaces use a consistent energy outline

The environments should also change within a level. Each level can move through three short beats - introduction, escalation, and finale - using new silhouettes and landmarks so forward movement feels like travelling through a real place. For example, Level 1 begins on an open bridge, passes through a mirrored gallery, and ends at a tall elevator atrium.

Practical environment scope

- Build each level from a small modular kit rather than modelling one enormous scene.
- Create 5-8 reusable structural pieces per level, then vary their scale, rotation, spacing, lighting, and combinations.
- Use a skybox or lightweight distant geometry to make the world feel larger without adding collision or detailed models.
- Reserve the most detailed assets and dynamic lights for the playable route and major landmarks.
- Keep the floor path and interactive objects readable at running speed; background detail should never hide hazards.
- Reuse geometry and materials, pool repeated effects, and unload/dispose one level before loading the next.

- Let the elevator hide level loading and visually preview the next environment through windows, light colour, sound, and particles.

Custom shader plan

Create a custom vertex and fragment shader for the tower's energy material. Time and game-state uniforms drive flowing bands, edge glow, distortion, and colour changes. The shader appears on elevator conduits, anchor nodes, and the final core, so it is integrated into the world and mechanics rather than added at the end.

Possible second shader if time allows: a fracture/dissolve shader used when glass objects break or when a level unloads.

Feedback

- Glass shards, brief impact light, camera impulse, and layered sound confirm a hit.
- Reticle colour and sound distinguish breakable targets, switches, and invalid surfaces.
- Low sphere and low integrity states are communicated visually and through audio.
- Music intensity rises with combo and danger; elevators provide a short musical reset.

8. Technical approach

Suggested stack

- Three.js for rendering and scene graph.
- Vite for development and production builds, configured with `base: './'.`
- A lightweight physics library such as Rapier or Cannon-es for projectiles, collisions, and selected debris.
- GLTF/GLB for models, with compressed textures and modest geometry.
- Web Audio or a lightweight audio library for sound and music.

High-level modules

- `Game`: state machine for menu, playing, paused, transition, win, and loss.
- `LevelManager`: loads/unloads each level and disposes GPU resources.
- `PlayerController`: movement, integrity, spheres, and animation state.
- `CameraController`: rigs, blending, shake/impulse, and picture-in-picture.
- `ProjectileSystem`: pooling, firing, collisions, and lifetime.
- `DestructionSystem`: fracture patterns, shard pooling, effects, and scoring.
- `ObstacleSystem`: reusable obstacle behaviours and triggers.
- `AudioManager`: music states and spatial sound.
- `UIManager`: HUD, menus, loading, options, and credits.
- `PerformanceMonitor`: FPS display available during development.

Scene hierarchy example

```
GameScene
|-- Environment
|   |-- StaticGeometry
|   |-- DynamicObstacles
|   `-- Effects
|-- PlayerRoot
|   |-- Avatar
|   |-- AimOrigin
|   `-- PlayerLight
|-- CameraRig
|   |-- FirstPersonMount
|   |-- ChasePivot
|   `-- ActiveCamera
|-- Projectiles
`-- LevelTriggers
```

The team must be able to explain why objects are parented this way. For example, the camera follows the rig, the aim origin follows the player, and world obstacles remain independent.

9. Scope boundaries

Minimum viable game

- Three short but complete levels with their distinct mechanics.
- First-person, chase, orbit/cinematic, and orthographic camera use.
- Mouse aiming/shooting and keyboard movement.
- Projectile collisions, breakable glass, integrity, score, win/loss, and restart.
- One substantial custom shader.
- Menus, HUD, loading feedback, audio, credits, and a production build.

Stretch goals

- Picture-in-picture switch previews.
- Procedural obstacle combinations.
- Replay ghost or local high scores.
- Second custom shader.
- Advanced fracture geometry.
- Accessibility options beyond volume, sensitivity, and reduced camera shake.

Online multiplayer should not be part of the first plan. It adds risk without strengthening the core experience as much as polish, cameras, shaders, and a complete third level would.

10. Rubric coverage map

Rubric category	Planned evidence
Viewing (10%)	Animated 3D scenes, first/third-person, orbit, orthographic and cinematic views, camera blending, animated avatar, world- and camera-attached objects
Control & Playability (10%)	Keyboard movement, mouse aiming/shooting, clear objectives, physics, win/loss, meaningful movement in 3D
3D Effects (15%)	Multiple lights, shadows, reflection/refraction, transparent glass, skybox, particles, bump/normal maps, heat/energy effects
Shaders (10%)	Original animated energy vertex/fragment shader using time and game-state uniforms; optional dissolve shader
Gameplay & Experience (25%)	Coherent tower story, three distinct levels, escalating mechanics, responsive controls, audio, balance, replayable score/combo system

Rubric category	Planned evidence
Polish (10%)	Loading screen, pause/options, restart without refresh, credits, consistent UI, performance budget, reduced camera shake option
Innovation (10%)	Camera-driven hybrid gameplay, playable elevator transitions, gravity-shifting final arena, original shader and procedural challenge combinations
Trailer (10%)	Planned capture moments from all three levels, elevator reveal, camera transitions, shader/core finale, clear title and gameplay

11. Sprint 1 goal

Sprint 1 outcome: prove the core feel and technical foundation with a hosted vertical slice, not three unfinished levels.

Sprint 1 must deliver

- A repository and Vite/Three.js project that every member can run.
- A production build served over local HTTP with relative asset paths.
- An early upload test to the department server using one textured scene.
- A greybox Level 1 corridor with forward movement and three lanes.
- Mouse aiming and projectile throwing.
- At least one breakable glass target with collision, shards, sound, and score feedback.
- Sphere count, integrity, score, win/loss, and restart without page refresh.
- A first-person camera plus a working blend to a third-person chase camera.
- A simple animated avatar or placeholder model visible in third-person.
- A shader prototype driven by a time uniform.
- A basic HUD, pause menu, and credits placeholder.
- An FPS counter and a first performance measurement on ordinary hardware.

Sprint 1 demonstration script

- 1 Open the hosted or production-build version in Chrome.
- 2 Start the game from the menu.
- 3 Move with the keyboard and aim/shoot with the mouse.
- 4 Break a target and explain the collision and destruction feedback.
- 5 Show score, sphere use, a win/loss state, and restart without refreshing.
- 6 Blend from first-person to third-person and explain the camera hierarchy.
- 7 Show the animated shader and explain its inputs.
- 8 Present the three-level plan and identify what is unique about each level.
- 9 Show the credits register and confirm how external assets will be tracked.

12. Initial backlog and ownership

Assign one owner and at least one reviewer to every item. Ownership is not permanent; it exists to prevent work from being assumed to be someone else's job.

Workstream	Sprint 1 task	Acceptance test	Owner
Project setup	Vite + Three.js structure and shared coding conventions	Clean install, dev run, and production build on two machines	TBD

Workstream	Sprint 1 task	Acceptance test	Owner
Deployment	Configure relative paths and test an uploaded build	Published URL opens with no 404s in Chrome	TBD
Player	Auto-run, lane/strafe movement, integrity	Keyboard input is responsive and collisions reduce integrity	TBD
Aiming/projectiles	Reticle, mouse aim, pooled spheres	Clicking fires toward the reticle; sphere count updates	TBD
Destruction/physics	Breakable pane and shard effect	Hit reliably triggers collision, score, sound, and pooled shards	TBD
Cameras/avatar	First-person and chase rigs with blend	Camera transition is smooth and avatar is visible in chase view	TBD
Shader/effects	Animated energy shader prototype	Team can explain vertex/fragment stages and uniforms	TBD
UI/audio	HUD, pause, basic options, credits register	All screens work; game pauses and restarts without refresh	TBD
Level design	Level 1 greybox and Level 2/3 paper layouts	Each level passes the one-sentence distinction test	TBD
QA/performance	Test checklist and FPS baseline	Core loop tested in Chrome; known issues recorded	TBD

13. Risks and responses

Risk	Response
Three levels become copies with new colours	Lock the unique mechanic and challenge type for each level before asset production
Too much time is spent on realistic fracture physics	Use authored fracture pieces and object pooling; prioritise impact feel over simulation complexity
Camera changes make obstacles unfair or cause motion sickness	Use section-specific camera permissions, smooth blends, clear sightlines, and a reduced-shake option
Physics or particles lower the frame rate	Pool projectiles/shards, cap active debris, reuse materials/geometries, and profile from Sprint 1
Level changes leak GPU memory	Centralise level disposal and test a full three-level replay repeatedly
Works locally but fails on the server	Use relative paths, lowercase asset names, production builds, local HTTP tests, and early deployment
External work is not credited	Maintain a credits register from day one with source URL, author, licence, and modifications
Team contributions become unclear	Track issues, commits, reviews, meeting decisions, and individual work throughout the project

14. Documentation set

Maintain these lightweight documents in the repository:

- 1 Project brief / game design document: this document, updated when the concept changes.
- 2 Technical design: architecture, scene hierarchy, state flow, asset pipeline, physics approach, shader explanation, and deployment rules.
- 3 Level design sheets: map, camera zones, mechanics, obstacles, assets, success/failure conditions, and performance budget for each level.
- 4 Sprint backlog: task, owner, reviewer, acceptance test, status, and dependencies.
- 5 Decision log: date, decision, alternatives, reason, and people present.

- 6 Credits and asset register: asset/library, creator, source, licence, modifications, and where it is used.
- 7 Test plan and bug log: browser, machine, build, test case, expected/actual result, severity, owner, and status.
- 8 Contribution evidence: issues, commits, reviews, authored assets, meeting notes, and demonstrations for each member.
- 9 Trailer and devlog plan: shot list, narration points, capture owner, editing owner, credits, and deadline.

15. Questions for the next group meeting

- Confirm the working title and one-sentence pitch.
- Agree on exactly which Smash Hit mechanics and visual features are inspirations and which elements the team will change to establish an original identity.
- Choose whether movement in Level 1 is discrete lanes or free horizontal strafing.
- Decide how many spheres and integrity points the player starts with.
- Approve the Level 3 gravity mechanic before detailed implementation.
- Choose a physics library after a small projectile/collision comparison.
- Record team member names, strengths, availability, and Sprint 1 owners.
- Obtain the exact alpha/check-in date and venue from Moodle.
- Agree on a shared asset style and what the team will create itself.

16. Definition of done for Sprint 1

Sprint 1 is done when the production build can be opened over HTTP in Chrome, a player can complete or fail the vertical slice using keyboard and mouse, the game restarts without a refresh, two camera rigs blend correctly, the custom shader prototype is visible and explainable, the team can state the unique mechanic of all three planned levels, and every external resource used so far appears in the credits register.

Project details to complete

- Module: COMS3006A / COMS3025A
- Team name: TBD
- Team members: TBD
- Mentor: TBD
- Alpha/check-in date: confirm on Moodle
- Beta date: confirm on Moodle
- Final date: confirm on Moodle
- Repository URL: TBD
- Hosted build URL: TBD